

Mochi Manatee App: Streamlit Dashboard

Documentation

Hudson Miller

Summary

This app was programmed in Visual Studio Code using Python. It uses Pymongo to connect Python files to MongoDB (the platform that hosts this project's database), Pandas to format the telemetry data taken from MongoDB into tables, and Utilities.py (from the backend of the Manatee Simulation project, specifically the RefactoredManateeAnalysis repository on GitHub) to process the data.

Streamlit is the service this app runs on and is essential to the app's functionality; currently, the app is hosted through Streamlit's Community Cloud (a free resource), but it can also be hosted locally. Port-forwarding is required for other users to access your local app, however, and will not be explained in this document.

<i>Component</i>	<i>Role</i>
Pymongo	Establishes a connection to MongoDB
Pandas	Formats telemetry data into tables
Utilities.py	Helper functions that process data before it is displayed

MongoDB Connection

You will need a URI to access the MongoDB database. Here, the URI is taken from Streamlit secrets, which can be found in the "Manage App" section of the dashboard when the app is opened. Click "Manage App," then the three vertical dots, then "Settings," then "Secrets;" you will be able to access the URI here if needed (i.e. it has to be changed due to a data breach).

WARNING: Making the URI or any sort of MongoDB access link public / unencrypted in the repository will notify anyone managing the database that their credentials have been compromised. I know this from experience, and it was very embarrassing. Keep access credentials in a .TOML file or in Streamlit secrets.

Then, the app creates a MongoClient object and connects to the ManateeSegments database and TelemetryCollection collection under it.

[st.set_page_config] is used to change the layout of the page.

[st.title] and [st.write] both act as “print” functions for Streamlit, the only differences being size and boldness.

[uri], [client], [db] and [telemetry] should all be self-explanatory.

```
28
29 # - - - - -
30 # MongoDB Connection
31 # - - - - -
32
33 # Get the URI from Streamlit's encrypted secrets list. This can be found in the "Manage App" menu.
34 # WARNING: Do NOT put the unencrypted credentials / access link anywhere in the repository.
35 # It will compromise the security of the database and make you look silly.
36 uri = st.secrets["MONGODB_URI"]
37 client = MongoClient(uri)
38
39 st.set_page_config(layout="wide")
40 st.title("Manatee Simulation Connection")
41 st.write("")
42
43 # Name of the database, then the name of the collection under that database.
44 db = client["ManateeSegments"]
45 telemetry = db["TelemetryCollection"]
46
```

Data Retrieval

Next, the app builds a dictionary of lists to store the telemetry data. It uses the query [telemetry.find()] to return all documents present in the database's collection, and in this case, appends them to the “sessions” list in order of session ID.

The app relies on [Utilities.parse_session_mongodb()] to parse telemetry data, which it then takes, adds names for each column, then puts into rows that are appended to the “summary_rows” list. Events like “playerBreathe” were used in an earlier, separate table that

displayed those metrics, but in the current table they're grouped together under "event_counts" to give more priority to reading times and scene times.

"summary_rows" is turned into the Pandas dataframe "summary_df" so that it can be displayed as a table using [`st.dataframe()`] and a bar chart using [`st.bar_chart()`].

Streamlit Interface

[`st.download_button()`] is a function built-in to Streamlit that creates a button which lets users download data, in this case, a .CSV file containing the "summary_df" table. Again, the fields within should be self-explanatory (a major benefit of Streamlit is its ease of use): "label" is the text displayed on the button, "data" is the data to be downloaded, "file_name" is the default name the file will have when downloaded, and "mime" sets the MIME type of the data.

```
107
108     # "download_button" is from Streamlit itself.
109     st.download_button(
110         label="Download CSV",
111         data=csv,
112         file_name="mongodb_export.csv",
113         mime="text/csv",
114     )
115
```

[`st.dataframe()`] displays Pandas dataframes. The reason I used it over a regular Streamlit table is because it automatically lets users sort the data by clicking on the column headers, type out search queries by clicking the upper-right corner of the table, and adds other quality of life additions as well. I added "width=stretch" so that the table would automatically widen to the size of the app's other elements.

```
121
122  st.dataframe(summary_df, width="stretch")
123
```

[st.selectbox()] creates a simple text box that, when clicked, displays a drop-down menu containing all session IDs present in the collection for users to choose from (“options=summary_df[“Session ID”]”), which will then be used to display the correct session as a bar chart.

```
132
133 # A dropdown box that will let users select which session to display on the chart via Session ID.
134 selected_session = st.selectbox(
135     "Select Session",
136     options=summary_df["Session ID"]
137 )
138
```

For the scene times chart, only columns with “Total time (ms)” are displayed. For the slide reading times chart, only columns with “Reading time (ms)” are displayed. These results retrieved using [session.filter()].

Finally, [st.bar_chart()] is the function that creates and displays bar charts. In this case, “readingTimes” is the data being displayed, “Slide” and “Time (ms)” serve as the X and Y axes respectively, and the “height=400” sets the height of the chart to 400 pixels.

```
157
158 # Second bar chart, displays only data that contains "Reading time (ms)"
159 st.write("Slide Reading Times")
160 readingTimes = session.filter(regex="Reading time \\(ms\\)")
161 readingTimes = readingTimes.rename_axis("Slide").reset_index(name="Time (ms)")
162
163 st.bar_chart(readingTimes, x="Slide", y="Time (ms)", height=400)
164
```

That should cover everything you need to know about the app. You can find the information on Utilities.py in the documentation for the Mochi Manatee Backend. Other resources, including the ones I used to build this app, can be found below.

Resources

Streamlit Mini Course by Tech With Time on YouTube:

<https://youtu.be/o8p7uQCGD0U?si=YpmsdGH0VM8p2jC9>

Streamlit documentation:

<https://docs.streamlit.io/>

Build a Movie Picker App with Streamlit and MongoDB | Fun Python Project! by Turtle Code on YouTube:

(shows the very basics of how to make a Streamlit app with MongoDB)

<https://youtu.be/ZBT7Cq1jQ7c?si=qQEMROqkKF0Kb9ga>

Miscellaneous Notes

Remember to have the correct version of Python, Git and Pip installed, along with the app's other requirements. If this method of connecting to MongoDB doesn't work, there is also a MongoDB extension for Visual Studio Code that can be found in the extensions menu, but I wasn't able to get that working. The MongoDB website itself also has alternative methods of connection, such as through Shell script or the Windows terminal.